Ders 9 Genişletilmiş Notu: Bağlamdan Bağımsız Diller İçin Pompalama Önsavı, Gramerlerin Sadeleştirilmesi, Chomsky ve Greibach Normal Biçimleri

Ders 9 Genişletilmiş Notu: Bağlamdan Bağımsız Diller İçin Pompalama Önsavı, Gramerlerin Sadeleştirilmesi, Chomsky ve Greibach Normal Biçimleri

Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Düzenli Dillerden Bağlamdan Bağımsız Dillere: Yeni Bir Pompalama Önsavı
- 2. Önsavın İfadesi
- 3. Üç Sayım Sezgisi
- 4. Örnek: $L = \{0^i 1 0^i 1 0^i\}$ Bağlamdan Bağımsız Değildir
- 5. Karşıt Örnek: $L_1 = \{0^i 1 0^i\}$ ve "Bu Bir İspat Değildir" Uyarısı
- 6. İspatın Sonu: Çelişkiyi Açıkça Yazmak
- 7. İki Şey Eşleşiyor Gibi Görünen Bir Dil: $L_2 = \{ww\}$
- 8. Benzer Görünen Ama Bağlamdan Bağımsız Olan: $L_3 = \{ww^R\}$
- 9. Gramerlerin Sadeleştirilmesi: Yerine Koyma
- 10. Boşa Giden Değişkenlerin Kaldırılması
- 11. Birim Üretimlerin Kaldırılması
- 12. Yararsız Üretimlerin Kaldırılması
- 13. Chomsky Normal Biçimi
- 14. Genel Olarak CNF'ye Dönüştürme
- 15. CNF Ne İşe Yarar?
- 16. Greibach Normal Biçimi
- 17. GNF'ye Dönüştürme
- 18. GNF ile CNF'nin Karşılaştırılması

1. Düzenli Dillerden Bağlamdan Bağımsız Dillere: Yeni Bir Pompalama Önsavı

Bağlamdan bağımsız diller (context-free languages, CFL) üzerinde çalışıyorduk ve son olarak belirsizliği (ambiguity) gördük. Bu derste önce bağlamdan bağımsız diller için pompalama önsavını (pumping lemma) göreceğiz.

Pompalama önsavını daha önce görmüştük. O zaman ona yalnızca "pompalama önsavı" demiştik, ama o aslında düzenli diller (regular languages) içindi. Şimdi aynı fikrin yeni bir sürümünü göreceğiz. Bu sürüm bağlamdan bağımsız diller içindir. Uygulaması da düzenli dillerdeki önsavın uygulamasına çok benzer.

2. Önsavın İfadesi

Hoca önsavı tahtaya şöyle yazdı:

Her bağlamdan bağımsız L dili için bir n tam sayısı vardır öyle ki, L'deki uzunluğu n'den büyük her z dizgisi için

$$z = uvwxy$$

bölünmesi vardır ve bu bölünme şu üç koşulu sağlar:

1. $|vwx| < n$,
2. $|vx| > 0$,
3. her $i > 0$ için $uv^i wx^i y \in L$.

Önsavdaki n sayısına pompalama sabiti ya da pompalama uzunluğu (pumping constant, pumping length) denir.

Düzenli dillerdeki önsavdan en önemli fark, bu kez iki yerin pompalanmasıdır. v ve x parçaları aynı anda ve aynı sayıda çoğaltılır.

> **Ders sonrası not:** Hoca eşitsizlikleri tahtaya kesin olarak yazdı ve "her $i > 0$" dedi. Önsavın standart biçimi şudur: $|z| \geq n$ olan her $z \in L$ için $|vwx| \leq n$, $|vx| \geq 1$ ve **her** $i \geq 0$ için $uv^i wx^i y \in L$. Kesin eşitsizlikli biçim, sabit bir artırılarak standart biçimden elde edilir, yani yanlış değildir. Ama "$i > 0$" önsavı zayıflatır: $i = 0$ da önsavın parçasıdır ve aşağı pompalama (pumping down) denir. Ezberlerken standart biçimi kullanın.
>
> **Ders sonrası not:** Hoca ikinci koşulu sözlü olarak "wx'in uzunluğu sıfırdan büyük" diye okudu. Pompalanan parçalar v ve x olduğu için doğrusu $|vx| > 0$'dır. Altyazıdaki biçim bir okuma sürçmesidir.

Hoca önsavın ispatını derste yapmadı. Bu ispatı kitaptan kendiniz okumanızı istedi.

> **Hocanın vurgusu:** Önsavın ifadesini yazmanız gerekiyor. Bundan sonraki örneklerin hepsi bu ifadeye dayanır.

3. Üç Sayım Sezgisi

Örneklere geçmeden önce hoca bir sezgi verdi. Genel olarak bağlamdan bağımsız diller yalnızca iki şeyi eşleştirebilir. Üç sayımın birbirine eşit olması gerekiyorsa bunu yapamazlar.

$L = \{0^i 10^i 10^i \mid i > 1\}$ dilini düşünelim. Burada üç sayım eşleşmelidir. İlk blokta iki sıfır varsa, ikinci ve üçüncü blokta da iki sıfır olmalıdır. Bağlamdan bağımsız diller bunların ikisini eşleştirebilir, ama üçüncüyü eşleştiremez. Bu nedenle dilin bağlamdan bağımsız olmadığını önsavı kullanmadan kolayca görürsünüz. Fakat bu yalnızca bir sezgidir. İspat için pompalama önsavı gerekir.

4. Örnek: $L = \{0^i 10^i 10^i\}$ Bağlamdan Bağımsız Değildir

4.1 İspatın Kurulumu

İspat, düzenli dillerdeki gibi çelişkiyle (proof by contradiction) yapılır:

1. Çelişki elde etmek için L'nin bağlamdan bağımsız olduğunu varsayalım.
2. n pompalama sabiti olsun.
3. Bir desen seçelim. Desenin uzunluğu n'den büyük olmalıdır, yani desen bir şekilde n'yi içermelidir.

Çok doğal bir seçim, dildeki i'lerin yerine n koymaktır:

$$z = 0^n 10^n 10^n$$

Önsava göre bu dizgi $z = uvwxy$ biçiminde yazılabilir. Şimdi bölünmenin olası durumlarına bakmalı ve hiçbirinin pompalanamadığını göstermeliyiz.

> **Ders sonrası not:** Dilde $i > 1$ istendiği için z'nin L'de olması $n \geq 2$ gerektirir. Önsav bir n için geçerliyse daha büyük her sabit için de geçerlidir. Bu yüzden n'yi yeterince büyük almak ispatı bozmaz.

Bütün durumlarda dizgiyi şöyle düşünüyoruz: birkaç sıfır, bir 1, başka sıfırlar, bir 1, başka sıfırlar. u, v, w, x, y parçalarını buna bir şekilde yerleştirmeliyiz. Pompalanacak olan v ve x'tir. Asıl soru bunların nereye konacağıdır.

4.2 Durum 1: v ve x Birer 1 Olsun

v ve x'i iki 1'in üzerine koymayı deneyelim. Sorun şudur: Dilde 1'lerin sayısı sabittir ve 1 yalnızca bir kez geçebilir. 1'ler çoğaltılırsa dizgi dilden çıkar. Bu yüzden bu seçim pompalanamaz.

4.3 Durum 2: v İlk Blokta, x Son Blokta Olsun

Bu seçim umut verici görünür, çünkü iki farklı sıfır bloğu birlikte büyür. Ama bu bölünme zaten seçilemez. Önsava göre $|vwx|$, n'den küçük olmalıdır. Burada ise v ile x arasında ortadaki n sıfır ve iki tane 1 vardır, ayrıca uçlardaki sıfırlar da işin içine girer. Dolayısıyla vwx çok uzundur ve bu atama yapılamaz.

4.4 Durum 3: v, w, x Ortadaki Blokta Olsun

Şimdi vwx'i kısa tutalım. Ortadaki bloktan bir sıfırı v, bir sıfırı x seçelim, aradakiler de w olsun. Soldaki 0^n1 parçası u, sağda kalan 10^n parçası y olsun.

Hoca bu seçimde $|vwx| = n$ çıktığını fark etti ve tahtadaki "$< n$" koşulunu sağlamak için ortadaki bloktan bir sıfırı daha y'ye verdi. Yani y'yi büyüterek vwx'i kısalttı. Parçalar şöyle oldu (ara adım eklendi):

$$u = 0^n1, \quad v = 0, \quad w = 0^{n-3}, \quad x = 0, \quad y = 010^n$$

Şimdi $i = 2$ için pompalayalım. u değişmez, v bir sıfır, x bir sıfır daha ekler:

$$uv^2wx^2y = 0^n10^{n+2}10^n$$

Bu dizgi L'de değildir, çünkü dil bütün blokların eşit olmasını ister. Burada blok uzunlukları n, $n+2$ ve n'dir.

Hoca buradan şu genellemeyi çıkardı: vx yalnızca sıfırlardan oluşuyorsa, üçüncü durumdaki gibi, pompalanmış dizgi dilde değildir. Sorunun kökü de şudur: Bağlamdan bağımsız önsav yalnızca iki yeri pompalamaya izin verir. Bu dilde ise üç yeri birlikte pompalamak gerekir. Bunu yapamayız.

> **Hocanın vurgusu:** Bir desende üç sayım görürseniz, onun pompalanamayacağını ve bağlamdan bağımsız olmadığını varsayabilirsiniz.

> **Ders sonrası not:** Hoca tahtada pompalanmış dizgiyi uv^2wx^2z diye yazdı. Son parça z değil y'dir. Doğru yazım uv^2wx^2y'dir. Ayrıca $|vwx| = n$ standart önsavda izinlidir, yani ilk seçimdeki düzeltmeye gerek yoktu. Sonuç iki durumda da aynıdır.

> **Ders sonrası not:** Tahtadaki üç durum bütün bölünmeleri kapsamaz. Tam gerekçe şöyledir. vx bir 1 içeriyorsa $i = 2$ için 1'lerin sayısı 2'den büyük olur ve dizgi L'de değildir. vx yalnızca sıfırlardan oluşuyorsa, $|vwx| \leq n$ olduğu için vwx birinci ve üçüncü bloğa aynı anda değemez, çünkü bu iki blok arasında $n+2$ sembol vardır. O zaman en az bir blok değişmez, en az bir blok büyür ve bloklar eşit kalmaz. İki durumda da pompalanmış dizgi L'de değildir. Python ile $n = 2$'den $n = 8$'e kadar bütün bölünmeler tarandı. Hiçbiri $i = 2$ sınavını geçmedi.

5. Karşıt Örnek: $L_1 = \{0^i 10^i\}$ ve "Bu Bir İspat Değildir" Uyarısı

Üç sayımı gördük. İki sayımda durum ne? Hoca şu dili yazdı:

$$L_1 = \{0^i 10^i \mid i > 1\}$$

Soru: Bu dil bağlamdan bağımsız mıdır? **Cevap:** Evet, çünkü eşleşmesi gereken iki şey vardır: soldaki sıfırlar ve sağdaki sıfırlar.

Önsavı deneyelim. $z = 0^n 10^n$ seçelim. Burada n pompalama sabitidir. Bölünmeyi şöyle yapalım: Ortadaki 1 parçası w olsun. 1'in hemen solundaki sıfır v, hemen sağındaki sıfır x olsun. Soldaki kalan sıfırlar u, sağdaki kalan sıfırlar y olsun:

$$u = 0^{n-1}, \quad v = 0, \quad w = 1, \quad x = 0, \quad y = 0^{n-1}$$

$vwx = 010$ olduğu için $|vwx| < n$ koşulu sağlanır. $i = 2$ için

$$uv^2wx^2y = 0^{n+1}10^{n+1}$$

elde edilir ve bu dizgi dildedir. Yani bu dizgi sorunsuz pompalanır. Eşleştirdiğimiz şey iki sayımdır ve bağlamdan bağımsız diller iki sayımı eşleştirebilir.

> **Ders sonrası not:** Hoca burada da önce sonuncu parçaya z dedi, sonra kendisi y olarak düzeltti. Doğrusu y'dir, çünkü z bütün dizginin adıdır. Bu bölünme Python ile $n = 3$'ten $n = 7$'ye kadar ve $i = 0, \ldots, 4$ için denendi. Pompalanan dizgilerin hepsi L_1'dedir.

Ancak bu örnek, L_1'in bağlamdan bağımsız olduğunun ispatı değildir. Bunu ispatlamak için bir bağlamdan bağımsız gramer (context-free grammar, CFG) vermek gerekir. Bu pompalama yalnızca bir örnektir.

> **Hocanın vurgusu:** Size "L_1'in bağlamdan bağımsız olduğunu ispatlayın" dersem ve bu pompalamayı yazarsanız, bu ispat olmaz. Pompalama önsavı, bir dilin bağlamdan bağımsız **olmadığını** ispatlamak için kullanılır. Bağlamdan bağımsız olduğunu göstermek için gramer verilir.
>
> **Ek bilgi (derste anlatılmadı):** Bu ders için istenen gramerler kısadır. L_1 için $S \to 0S0 \mid 00100$, $L_3 = \{ww^R\}$ için $S \to 0S0 \mid 1S1 \mid \varepsilon$ yeterlidir. Önsav yalnızca gerekli bir koşul verir: Bağlamdan bağımsız her dil pompalanır, ama pompalanan her dil bağlamdan bağımsız olmak zorunda değildir. Hocanın uyarısının mantıksal nedeni budur. İki gramerin ürettiği dil, sınırlı uzunluğa kadar tam taramayla Python'da denendi.

6. İspatın Sonu: Çelişkiyi Açıkça Yazmak

Hoca burada L için başladığı ispata döndü, çünkü çelişkiyi henüz yazmamıştı. İspatın son satırları şöyledir:

- Bütün durumlarda, pompalanabilen bir $z = uvwxy$ bölünmesi bulamıyoruz.
- Bu bir çelişkidir.
- Demek ki L'nin bağlamdan bağımsız olduğu varsayımı yanlıştır.
- L bağlamdan bağımsız değildir.

Önsavın ispat kalıbı böylece tamamlanır: varsay, sabiti al, n'ye bağlı dizgi seç, bütün bölünmeleri incele, çelişkiyi yaz.

7. İki Şey Eşleşiyor Gibi Görünen Bir Dil: $L_2 = \{ww\}$

Eşleşmesi gereken iki şey varsa dil bağlamdan bağımsız olabilir. Yine de dikkatli olmak gerekir, çünkü iki şey eşleşiyor gibi görünse bile dil pompalanamayabilir. Hoca L_1 ile karşılaştırmak için şu dili yazdı:

$$L_2 = \{ww \mid w \in (0+1)^*\}$$

Bu dil bağlamdan bağımsız değildir. İlk bakışta iki şeyi eşleştiriyor gibi görünür. Ama üzerinde düşününce eşleşmesi gereken şeylerin ikiden fazla olduğu anlaşılır.

Hoca n pompalama uzunluğu olmak üzere şöyle bir dizgi seçilebileceğini söyledi:

$$z = 0^n 1^n 0^n 1^n$$

Bu dizgi L_2'dedir, çünkü $w = 0^n 1^n$ alınırsa $z = ww$ olur. Önsava göre pompalanabilmesi gerekir, ama pompalanamaz. Hoca bunu kendiniz kontrol etmenizi istedi.

> **Ders sonrası not:** Python ile $n = 1$'den $n = 8$'e kadar $z = 0^n 1^n 0^n 1^n$ için bütün bölünmeler tarandı. $|vwx| \leq n$ ve $|vx| \geq 1$ koşullarını sağlayan hiçbir bölünme $i = 0$ ya da $i = 2$ için dilde kalmadı. Buna karşılık $z = 0^n 10^n 1$ seçilirse $n \geq 3$ için dizgi pompalanabilir: v ilk bloğun son sıfırı, $w = 1$, x ikinci bloğun ilk sıfırı alınırsa pompalanmış dizgi $0^{n-1+i}10^{n-1+i}1$ olur ve yine ww biçimindedir. Bu yüzden L_2 ispatında dizgi seçimi önemlidir: $0^n 10^n 1$ çelişki vermez, $0^n 1^n 0^n 1^n$ verir.

8. Benzer Görünen Ama Bağlamdan Bağımsız Olan: $L_3 = \{ww^R\}$

Hoca karşılaştırma için bir dil daha verdi:

$$L_3 = \{ww^R \mid w \in (0+1)^*\}$$

Bu dil, bir w dizgisi ile onun tersinin (reverse) art arda yazılmasından oluşur ve bağlamdan bağımsızdır. Hoca bu dildeki dizgilerin pompalanabildiğini kendiniz denemenizi istedi ve burada durdu.

> **Ders sonrası not:** Deneme için en doğal bölünme, ww^R'nin tam ortasındaki iki sembolü v ve x almaktır. Bu iki sembol birbirinin aynısıdır ve w parçası boştur. Her $i \geq 0$ için dizgi yine bir çift uzunluklu palindromdur. Python ile uzunluğu 14'e kadar bütün dizgilerde, $i = 0, \ldots, 4$ için denendi. Önceki uyarı burada da geçerlidir: Bu pompalama, L_3'ün bağlamdan bağımsız olduğunun ispatı değildir.

9. Gramerlerin Sadeleştirilmesi: Yerine Koyma

Dersin ikinci konusu bağlamdan bağımsız gramerlerin sadeleştirilmesidir (simplification of context-free grammars). Gramerlerde bazen gereksiz üretimler (productions) olur. Bunları bazı yerine koymalarla (substitution) sadeleştirebiliriz.

9.1 Örnek

Hocanın örneği şu gramerdir:

$$S \to aB, \quad A \to aaA, \quad A \to abBc, \quad B \to aA, \quad B \to b$$

Birinci adım: $B \to b$ **yerine konur.** Bu üretimi artık görmeyeceğiz. Bunun için sağ taraflarda büyük B arıyoruz. Bulduğumuz yerde B'yi küçük b ile değiştirdiğimiz **yeni bir kural ekliyoruz**. Var olan kuralı silmiyoruz, çünkü B'nin hâlâ başka bir üretimi var:

$$S \to aB \mid ab, \quad A \to aaA, \quad A \to abBc \mid abbc, \quad B \to aA$$

Bu yeni gramer ilk gramere denktir.

> **Ders sonrası not:** Hoca $B \to b$ için "birim üretim" (unit production) dedi. Bu ifade dersin devamındaki tanımla uyuşmaz. Birim üretim, bir değişkenin tek bir **değişken** ürettiği kuraldır ($X \to Y$). $B \to b$ ise tek bir terminal üreten sıradan bir kuraldır.

İkinci adım: $B \to aA$ yerine konur. Sol tarafta artık yalnız bir B kuralı kaldı. Bu yüzden B'den tamamen kurtulabiliriz. Sağ taraflardaki iki B'yi aA ile değiştiriyoruz. B'nin başka üretimi kalmadığı için bu sefer eski kuralları tutmuyoruz:

- $S \to aB \mid ab$ kuralı $S \to aaA \mid ab$ olur.
- $A \to aaA$ kuralında yalnız A'lar vardır. Değişiklik yapılmaz.
- $A \to abBc \mid abbc$ kuralı $A \to abaAc \mid abbc$ olur.

Son gramer şudur:

$$S \to aaA \mid ab, \quad A \to aaA \mid abaAc \mid abbc$$

Dili aynı tuttuk, ama değişken sayısı üçten ikiye indi. Üç gramerin aynı dili ürettiği, uzunluğu 16'ya kadar bütün dizgiler üzerinde Python ile doğrulandı.

9.2 Genel Kural

Hoca genel kuralı iki parça olarak yazdı.

- $A \to xBz$ ve $B \to y_1$ varsa (y_1 terminallerden oluşuyorsa), yerine koyma $A \to xBz \mid xy_1z$ verir. Yani yeni kural eklenir.
- $A \to xBz$ ve $B \to aC$ varsa, sağ tarafta değişken bulunduğu için yeni kural eklemek yerine $A \to xaCz$ yazılır.

Soru: Genel kuralın ilk örneğinde $A \to xBz$'yi silmemiz gerekmiyor mu? B'yi kaldırdık. **Cevap:** Evet, bu örnekte yalnız bir tane B üretimi olduğu için silmemiz gerekir, haklısın. Ama $B \to aC$ gibi başka bir B üretimi de olsaydı silmezdik, çünkü o zaman hâlâ bir B var demektir.

Bu sorudan çıkan ilke şudur: Elde ettiğimiz gramer, orijinal gramerle aynı dizgileri üretebilmelidir.

> **Ders sonrası not:** Hoca, kuralın eklenip eklenmemesini yerine konan üretimin terminal ya da değişken içermesine bağladı. Asıl ölçüt bu değildir, B'nin başka üretiminin kalıp kalmamasıdır. B'nin bir üretimi silinince, B'nin geçtiği her yere o üretimin sağ tarafı eklenir. B'nin başka üretimi kalıyorsa eski kural durur, kalmıyorsa eski kural da silinir. Öğrencinin sorusu ve hocanın cevabı bu ölçütle uyumludur.

Bu yerine koymaları bundan sonraki sadeleştirmelerde kullanacağız.

10. Boşa Giden Değişkenlerin Kaldırılması

$X \to \varepsilon$ biçimindeki bir kurala boş üretim (null production) denir. Burada X boş dizgiyi (null) üretir ve boşa giden bir değişkendir (nullable variable). Daha genel olarak, bir Y değişkeni birkaç adım sonunda boş dizgiye varıyorsa, yani $Y \Rightarrow^* \varepsilon$ ise, Y de boşa gidendir. Genel olarak boşa giden değişkenlerden kurtulmak isteriz.

Örnek gramer:

$$S \to aMb, \quad M \to aMb, \quad M \to \varepsilon$$

Burada M açıkça boşa gidendir. M yerine ε koyacağız. Sol taraftaki M'lere dokunmuyoruz, $M \to \varepsilon$ kuralı ise artık yazılmıyor. Sağ taraflarda M değişkeni hâlâ var olduğu için aMb kalır. Buna bir de M yerine boş dizgi konmuş kuralı ekleriz:

$$S \to aMb \mid ab, \quad M \to aMb \mid ab$$

İki gramer de $\{a^n b^n \mid n \geq 1\}$ dilini üretir. Bu, uzunluğu 16'ya kadar Python ile doğrulandı.

Bütün boş üretimler kaldırıldıktan sonra, başlangıç değişkeni dışındaki bütün boşa giden değişkenler ortadan kalkar. Değişkenin kendisi grameri terk etmek zorunda değildir: Örnekte M duruyor, ama artık boş dizgi üretemiyor.

Başlangıç değişkeni ise istisnadır. $S \to \varepsilon$ gibi bir kural varsa bu konuda yapabileceğimiz bir şey yoktur, çünkü başlangıç değişkenini başka bir şeyle değiştiremeyiz.

> **Ek bilgi (derste anlatılmadı):** Aynı boşa giden değişken bir sağ tarafta birden çok kez geçiyorsa, her geçişin silinmiş ve silinmemiş bütün birleşimleri için ayrı kural yazılır. Sağ tarafın tamamı boşalıyorsa bu kural eklenmez, çünkü yeni bir boş üretim yaratır. $S \to \varepsilon$'un kaldırılamamasının nedeni de dilin ε içermesidir.

11. Birim Üretimlerin Kaldırılması

Birim üretim (unit production), bir değişkenin tek bir değişken ürettiği kuraldır: $X \to Y$. İki tarafta da tek bir değişken vardır. Hoca bunun gereksizliğini şöyle anlattı: X doğrudan Y üretiyorsa X yerine Y kullanabiliriz. Neden iki ayrı değişken tutalım?

Örnek gramer:

$$S \to aA, \quad A \to a, \quad A \to B, \quad B \to A, \quad B \to bb$$

$A \to B$ ve $B \to A$ birim üretimdir. Bunları tek tek kaldıracağız.

Birinci adım: $A \to B$ kaldırılır. Sağ tarafta A gördüğümüz her yere B'li sürümü de ekliyoruz. Değişiklikler yalnız sağ tarafta yapılır, sol taraftaki A'lara dokunulmaz. A değişkeni hâlâ durduğu için A'lı kuralları silmiyoruz:

- $S \to aA$ kuralı $S \to aA \mid aB$ olur.
- $A \to a$ aynı kalır.
- $A \to B$ yazılmaz.
- $B \to A$ kuralı $B \to A \mid B$ olur.
- $B \to bb$ aynı kalır.

İkinci adım: $B \to B$ silinir. $X \to X$ biçimindeki üretimler hiçbir yerine koyma gerektirmeden hemen silinebilir. Grameri bir kere daha yazalım:

$$S \to aA \mid aB, \quad A \to a, \quad B \to A, \quad B \to bb$$

Üçüncü adım: $B \to A$ kaldırılır. Çünkü bu hâlâ bir birim üretimdir. Sağ tarafta B gördüğümüz yere B yerine A yazılmış sürümü ekliyoruz. B değişkeni hâlâ durduğu için aB kalır:

$$S \to aA \mid aB \mid aA, \quad A \to a, \quad B \to bb$$

Dördüncü adım: Tekrar eden kural silinir. İki aA aynıdır, biri atılır. Son gramer şudur:

$$S \to aA \mid aB, \quad A \to a, \quad B \to bb$$

Bu ara gramerlerin ve son gramerin hepsi $\{aa, abb\}$ dilini üretir. Python ile doğrulandı.

Soru: $B \to A$ silindi ve A küçük a üretiyor. O hâlde $B \to bb$'nin yanında bir de küçük a olması gerekmiyor mu? **Cevap:** Gerekmiyor. Bu değişiklikleri sağ tarafta yapıyoruz, yani sağda bir B görürsek onu değiştiriyoruz. Sol tarafa dokunmuyoruz. $B \to bb$ kuralının sağında büyük B yok. B'nin sağda geçtiği yer $S \to aB$'dir, onu değiştirince aA çıkıyor, o da zaten var olduğu için atılıyor.

> **Ders sonrası not:** Cevap doğrudur. Son gramerde B artık a üretmez, ama $S \to aA$ kuralı aa dizgisini korur. Öğrencinin aklındaki $B \to bb \mid a$ yazımı da aynı dili verir. Python ile denendi: İki gramer de $\{aa, abb\}$ üretir. Bu ikinci yol, sol taraftaki değişkene Y'nin kurallarını kopyalama yoludur.

> **Ders sonrası not:** Dersteki yöntem, kaldırılan birim üretimin sol tarafı başlangıç değişkeni ise tek başına yetmez. Türetme doğrudan S ile başladığı için, sağ taraflara sürüm eklemek $S \to Y$ kuralının silinmesini telafi etmez ve Y'den gelen dizgiler kaybolur. Bu durumda Y'nin kuralları S'ye kopyalanmalıdır. Python ile $S \to A$, $A \to a$ grameriyle denendi: Yalnız silme yapılırsa a dizgisi kaybolur.

12. Yararsız Üretimlerin Kaldırılması

Kurtulacağımız üçüncü tür yararsız üretimlerdir (useless productions).

12.1 Bitmeyen Türetmeler

Bazı türetmeler hiç bitmez:

$$S \Rightarrow aA \Rightarrow aaA \Rightarrow aaaA \Rightarrow \ldots$$

Böyle bir türetme sonsuza kadar sürer. Bir terminal dizgiyle sonuçlanmadığı için gereksizdir. Hocanın gösterdiği gramer şudur:

$$S \to aSb, \quad S \to \varepsilon, \quad S \to A, \quad A \to aA$$

Sorun $A \to aA$ kuralındadır. A'nın hiçbir terminal üretimi yoktur, "A, b üretir" gibi bir kural yoktur. Hoca bunu türetme ağacı üzerinden anlattı: S'den başlayıp A'ya inen dallar, A'dan yine A çıktığı için aşağıya hiç varamaz. Dibini bulamazsınız, yapraklara ulaşamazsınız. Bu türetme adımları bize hiçbir şey vermez. Bu yüzden bunları bulup atmak isteriz. Bunun için iki adım vardır.

12.2 Adım 1: Dizgi Üretebilen Değişkenleri Bulmak

Hoca bu noktada örneği değiştirdi:

$$S \to aS \mid A \mid C, \quad A \to a, \quad B \to aa, \quad C \to aCb$$

Birinci adımda değişkenleri tarıyoruz ve yalnız terminal ya da boş dizgi üreten dizgilere varabilen değişkenleri buluyoruz. Bunlara yararlı (useful) değişkenler diyelim ve U kümesine koyalım. Bu işi turlarla (round) yapabiliriz:

- **1. tur:** Üretiminin sağ tarafı yalnız terminal olan değişkenler seçilir. $A \to a$ ve $B \to aa$ olduğu için $U = \{A, B\}$.
- **2. tur:** Üretiminin sağ tarafında yalnız yararlı değişkenler ve terminaller bulunan değişkenler eklenir. $S \to A$ olduğu için S yararlıdır. Sağ taraf doğrudan A olmak zorunda değildir, A içeren bir dizgi de olabilir. $U = \{S, A, B\}$.

Küme değişmeyene kadar böyle devam edilir. C hiçbir zaman kümeye girmez, çünkü C'nin tek kuralı yine C içerir.

Şimdi U dışındaki değişkenleri içeren üretimleri siliyoruz. Dört değişkenden üçü yararlı, biri yararsızdır. $S \to aS$ kalır, çünkü S yararlıdır. $S \to A$ kalır, çünkü A yararlıdır. $S \to C$ ve $C \to aCb$ gider. Sonuç:

$$S \to aS \mid A, \quad A \to a, \quad B \to aa$$

12.3 Adım 2: Ulaşılabilir Değişkenleri Bulmak

Hâlâ yapacak işimiz var. Sorun B'dir. S'den başlarsak B'ye asla ulaşamayız. Bu kez S'den başlayıp takip edeceğiz ve ulaşılamayan değişkenlerden kurtulacağız.

Bunun için bağımlılık grafiği (dependency graph) kullanırız. Düğümler değişkenlerdir. Bir değişkenin kuralının sağ tarafında başka bir değişken geçiyorsa, ondan ona kenar çizilir:

- $S \to aS$ nedeniyle S'den S'ye kenar vardır.
- $S \to A$ nedeniyle S'den A'ya kenar vardır.
- A değişken üretmez, yalnız terminal üretir. Ondan kenar çıkmaz.
- B grafikte durur, ama ona gelen kenar yoktur. B ulaşılamaz.

Yalnız başlangıç değişkeni S'den ulaşılabilen değişkenleri tutarız. $B \to aa$ gereksizdir. Son gramer:

$$S \to aS \mid A, \quad A \to a$$

Üç gramerin dili aynıdır ($\{a^k \mid k \geq 1\}$). Bu, uzunluğu 12'ye kadar Python ile doğrulandı. Hocanın ilk gramerinde de aynı işlem A'yı atar ve $S \to aSb \mid \varepsilon$ bırakır (ara adım eklendi). Bu da Python ile doğrulandı.

> **Ek bilgi (derste anlatılmadı):** Dersteki sıra önemlidir. Önce terminal üretemeyen değişkenler, sonra ulaşılamayan değişkenler silinir. Birinci adımdaki silmeler yeni ulaşılamayan değişkenler yaratabilir. Sıra tersine çevrilirse sonda yararsız değişken kalabilir.

Hocanın özetiyle üç sadeleştirmeyi gördük: boş üretimlerin, birim üretimlerin ve yararsız değişken ile üretimlerin kaldırılması. Sıradaki adımda bunların üçünü birlikte kullanıp çok daha temiz biçimler elde edeceğiz.

13. Chomsky Normal Biçimi

Gereksiz şeylerden kurtulduktan sonra daha iyi biçimlere geçebiliriz. Bağlamdan bağımsız gramerler için bunlara normal biçimler (normal forms) denir. Birincisi Chomsky normal biçimidir (Chomsky normal form, CNF).

CNF'de her üretim şu iki türden biri olmalıdır:

$$A \to BC \qquad \text{ya da} \qquad A \to a$$

Yani bir değişken ya tam iki değişken üretir ya da tam bir terminal üretir. $A \to aA$ gibi bir kural, yani terminal ile değişkenin karışık olduğu bir kural, kabul edilmez.

Hoca bu biçimin daha çok makineler için yararlı olduğunu söyledi. Bir insan gözüyle okuduğunuzda daha okunaklı değildir. Ama bir bilgisayar programı için avantajları vardır. CNF'ye bilgisayarlar için dönüştürürsünüz.

13.1 Örnek

Hocanın örneği:

$$S \to ABa, \quad A \to aab, \quad B \to Ac$$

Bu üretimlerin hiçbiri istediğimiz biçimde değildir. Hepsinde ya uzun bir sağ taraf ya da karışık semboller vardır. Dolayısıyla gramer CNF'de değildir. Dönüştürme iki adımdır.

Adım 1: Terminaller için yeni değişkenler. Terminallerimiz a, b ve c'dir. Her biri için bir değişken tanımlıyoruz:

$$T_a \to a, \quad T_b \to b, \quad T_c \to c$$

Eski kuralları bu değişkenlerle yeniden yazıyoruz:

$$S \to ABT_a, \quad A \to T_aT_aT_b, \quad B \to AT_c$$

Şimdi hangi kurallar CNF'dedir? $T_a \to a$, $T_b \to b$, $T_c \to c$ tam bir terminal ürettiği için CNF'dedir. $B \to AT_c$ iki değişken ürettiği için CNF'dedir, T_c'nin de bir değişken olduğunu unutmayın. Geriye iki uzun kural kaldı: $S \to ABT_a$ ve $A \to T_aT_aT_b$.

Adım 2: Uzun kuralları bölmek için yeni değişkenler. Bu kurallarda ikiden fazla değişken var. Bir kısmını yeni bir değişkende birleştireceğiz:

- BT_a parçasına V_1 diyelim. Böylece $S \to AV_1$ ve $V_1 \to BT_a$ olur.
- T_aT_b parçasına V_2 diyelim. Böylece $A \to T_aV_2$ ve $V_2 \to T_aT_b$ olur.

Son gramer şudur:

Kural	Tür
$S \to AV_1$	iki değişken
$A \to T_aV_2$	iki değişken
$B \to AT_c$	iki değişken
$V_1 \to BT_a$	iki değişken
$V_2 \to T_aT_b$	iki değişken
$T_a \to a$	bir terminal
$T_b \to b$	bir terminal
$T_c \to c$	bir terminal

Bütün kurallar artık CNF'dedir. Python ile biçim denetlendi. Üç gramer de yalnız $aabaabca$ dizgisini üretir.

14. Genel Olarak CNF'ye Dönüştürme

Bu noktaya gelebilmek için aslında boşa giden değişkenleri ve birim üretimleri de kaldırmak gerekir. Genel sonuç şudur: Boş dizgi üretmeyen her bağlamdan bağımsız gramer için, CNF'de denk bir gramer elde edilebilir.

Genel işlem sırası:

1. Boşa giden değişkenleri kaldır.
2. Birim üretimleri kaldır.
3. Yararsız üretimleri kaldır. Bu adım isteğe bağlıdır, çünkü yararsız üretimler bulunsa bile gramer CNF'de olabilir.
4. Terminaller için yeni değişkenler ekle.
5. Uzun kuralları bölmek için yeni değişkenler ekle.

Başlangıç değişkeni boş dizgi üretiyorsa ne olur? Hoca, böyle bir grameri CNF'ye dönüştüremeyeceğimizi düşündüğünü söyledi.

> **Ders sonrası not:** Bu, kullanılan tanıma bağlıdır. Derste yazılan tanımda ($A \to BC$ ya da $A \to a$) boş dizgi üretilemez, bu yüzden dilde ε varsa bu tanımla dönüştürme yapılamaz. Bazı kitaplardaki tanım ise yalnız başlangıç değişkeni için $S \to \varepsilon$ kuralına izin verir ve S'nin hiçbir sağ tarafta geçmemesini ister. O tanımda ε içeren dillerin de CNF grameri vardır. Hangi tanımın kullanıldığına dikkat edin.

15. CNF Ne İşe Yarar?

CNF, ayrıştırma (parsing) ve teorem ispatları için iyidir. Bu yüzden CNF'ye dönüştürürüz.

> **Ek bilgi (derste anlatılmadı):** CNF'de türetme ağacı ikili bir ağaçtır. Hocanın atladığı pompalama önsavı ispatı bu yapıdan yararlanır. Dizgi yeterince uzunsa ağaçta yeterince uzun bir yol vardır ve bu yolda bir değişken tekrar eder. Tekrar eden değişken arasındaki parça, aynı anda büyütülebilen v ve x'i verir.

16. Greibach Normal Biçimi

İkinci normal biçim Greibach normal biçimidir (Greibach normal form, GNF). Bu biçimde bütün üretimler şöyledir:

$$A \to aV_1V_2 \cdots V_k, \qquad k \geq 0$$

Yani sağ tarafta önce tek bir terminal gelir, ardından yalnız değişkenler gelir. k sıfır olabilir, dolayısıyla tek başına bir terminal de kabul edilir.

Ders sonrası not: Hoca adın okunuşundan emin olmadığını söyledi ve "Grainbach" benzeri okudu. Altyazıda da ad birkaç bozuk biçimde geçiyor. Doğru yazım Greibach'tır.

16.1 GNF'de Olan Bir Örnek

$$S \to cAB, \quad A \to aA \mid bB \mid b, \quad B \to b$$

Bu gramer GNF'dedir. Her kural terminal ve ardından değişkenler biçimindedir. $A \to b$ ve $B \to b$ kurallarında yalnız bir terminal vardır, değişken sayısı sıfırdır ve bu kabul edilir. Python ile biçim denetlendi.

16.2 GNF'de Olmayan Bir Örnek

$$S \to abSb, \quad S \to aa$$

Bu kurallar GNF'de değildir. İlk kuralda başlangıçta tek bir terminal var, bu doğrudur, ama hemen yanında başka bir terminal var. Sonda da bir terminal daha var. İkinci kuralda da ilk terminalin yanında ikinci bir terminal var. Baştaki terminal dışındaki her şey değişken olmalıdır.

17. GNF'ye Dönüştürme

Hoca aynı örnek üzerinden dönüştürmeyi gösterdi. Terminaller için geçici değişkenler ekleyip onları kullanabiliriz. b için T_b, a için T_a tanımlarız. Baştaki terminallere dokunmayız. Yeni gramer şudur:

$$S \to aT_bST_b, \quad S \to aT_a, \quad T_a \to a, \quad T_b \to b$$

Bu gramer GNF'dedir. Python ile biçim denetlendi ve uzunluğu 14'e kadar iki gramerin aynı dili ürettiği görüldü.

Ders sonrası not: Derste gösterilen yöntem yalnız her kural zaten bir terminalle başlıyorsa işe yarar. $S \to AB$ gibi değişkenle başlayan ya da $A \to Aa$ gibi sol özyinelemeli (left recursive) kurallarda önce sol özyineleme kaldırılmalı, değişkenler sıralanarak yerine koyma yapılmalıdır.

18. GNF ile CNF'nin Karşılaştırılması

Hoca elindeki bir notu okudu: GNF'ler dizgileri ayrıştırmak için çok iyidir, hatta CNF'den de iyidir. Ama bir gramerin GNF'sini bulmak, CNF'sini bulmaktan daha zordur.

Hoca bunun kendisine zor görünmediğini, kolay göründüğünü ekledi. Yine de böyle bir not olduğunu ve bunun çok da önemli olmadığını söyledi.

Ders sonrası not: Derste gösterilen örnekte dönüştürme kolaydır, çünkü kurallar zaten terminalle başlar. Nottaki "daha zordur" ifadesi, bir önceki bölümde anlatılan genel durum içindir.

Dersin Toparlaması

Bu derste önce bağlamdan bağımsız diller için pompalama önsavını gördük. v ve x olmak üzere iki yer birlikte pompalandığı için, bağlamdan bağımsız diller genel olarak iki sayımı eşleştirebilir. $\{0^i10^i10^i\}$ gibi üç sayım isteyen diller ve ilk bakışta iki şey eşleştiriyor gibi görünen $\{ww\}$ önsavla elenir. Önsav yalnız "bağlamdan bağımsız değildir" sonucu için kullanılır. Bağlamdan bağımsız olduğunu göstermek için gramer verilir.

Sonra gramerleri yerine koyma, boşa giden değişkenleri kaldırma, birim üretimleri kaldırma ve yararsız üretimleri kaldırma ile sadeleştirdik. Bu sadeleştirmeler Chomsky normal biçimine dönüştürmenin ilk adımlarıdır. Son olarak Greibach normal biçimini ve basit bir dönüştürme

örneğini gördük. Kitabın 2. bölümünde geriye yalnız aşağı itmeli otomatlar (pushdown automata) kaldı. Onları gelecek derste göreceğiz. Bölümün alıştırmalarının çoğunu artık yapabiliyor olmalısınız.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.